Nom i Cognoms:	Denis Fernández
URL Repositori Github:	

ACTIVITAT
Objectius: ● Familiaritzar-se amb el desenvolupament d'APIs REST utilitzant Express.js ● Aprendre a integrar serveis de processament de llenguatge natural i visió artificial ● Practicar la implementació de patrons d'accés a dades i gestió de bases de dades ● Desenvolupar habilitats en documentació d'APIs i logging ● Treballar amb formats JSON i processament de dades estructurades
Criteris d'avaluació: - Cada pregunta indica la puntuació corresponent
Entrega: - Repositori git que contingui el codi que resol els exercicis i, en el directori "doc", aquesta memòria resposta amb nom "memoria.pdf"

Punt de partida
https://github.com/jpala4-ieti/DAM-M0486-Tema3-RA6-PR4.2-Punt-Partida-25-26

Preparació de l'activitat

- Clonar el repositori de punt de partida
- Llegir els fitxers README*.md que trobaràs en els diferents directoris
- Assegurar-te de tenir una instància de MySQL/MariaDB funcionant
- Tenir accés a una instància d'Ollama funcionant (al centre te'n facilitem una)

Entrega

- URL de resopitori

Exercicis

Exercici 1 (2.5 punts)

L'objectiu de l'exercici és familiaritzar-te amb **xat-api**. Respon la les preguntes dins el requadre que trobaràs al final de l'exercici.

Configuració i Estructura Bàsica:

1. Per què és important organitzar el codi en una estructura de directoris com controllers/, routes/, models/, etc.? Quins avantatges ofereix aquesta organització?
2. Analitzant el fitxer server.js, quina és la seqüència correcta per inicialitzar una aplicació Express? Per què és important l'ordre dels middlewares?
3. Com gestiona el projecte les variables d'entorn? Quins avantatges ofereix usar dotenv respecte a hardcodejar els valors?

API REST i Express:

1. Observant chatRoutes.js, com s'implementa el routing en Express? Quina és la diferència entre els mètodes HTTP GET i POST i quan s'hauria d'usar cadascun?
2. En el fitxer chatController.js, per què és important separar la lògica del controlador de les rutes? Quins principis de disseny s'apliquen?
3. Com gestiona el projecte els errors HTTP? Analitza el middleware errorHandler.js i explica com centralitza la gestió d'errors.

Documentació amb Swagger:

1. Observant la configuració de Swagger a swagger.js i els comentaris a chatRoutes.js, com s'integra la documentació amb el codi? Quins beneficis aporta aquesta aproximació?
2. Com es documenten els diferents endpoints amb els decoradors de Swagger? Per què és important documentar els paràmetres d'entrada i sortida?
3. Com podem provar els endpoints directament des de la interfície de Swagger? Quins avantatges ofereix això durant el desenvolupament?

Base de Dades i Models:

1. Analitzant els models Conversation.js i Prompt.js, com s'implementen les relacions entre models utilitzant Sequelize? Per què s'utilitza UUID com a clau primària?
2. Com gestiona el projecte les migracions i sincronització de la base de dades? Quins riscos té usar `sync()` en producció?
3. Quins avantatges ofereix usar un ORM com Sequelize respecte a fer consultes SQL directes?

Logging i Monitorització:

1. Observant logger.js, com s'implementa el logging estructurat? Quins nivells de logging existeixen i quan s'hauria d'usar cadascun?
2. Per què és important tenir diferents transports de logging (consola, fitxer)? Com es configuren en el projecte?
3. Com ajuda el logging a debugar problemes en producció? Quina informació crítica s'hauria de loguejar?

Configuració i Estructura Bàsica

1. **Per què és important organitzar el codi en `controllers/`, `routes/`, `models/`, etc.?**
   Aquesta separació estableix responsabilitats clares: les *routes* exposen URLs i deleguen; els *controllers* contenen la lògica que resol la petició; els *models* defineixen l'estructura de dades i l'accés a la base de dades. Això facilita localitzar canvis, evitar duplicació, escriure tests i treballar col·laborativament sense trepitjar codi. A nivell pràctic permet evolucionar el projecte (afegir noves funcionalitats o substituir tecnologies) sense refactoritzar-ho tot.

2. **Quina és la seqüència correcta per inicialitzar una aplicació Express (segons `server.js`)?**
   Primer es carreguen les variables d'entorn, després es creen i configuren els middlewares generals (CORS, JSON, logging, Swagger) abans de registrar les rutes (`/api/chat`). Al final es posa el middleware d'errors i s'inicia el servidor i la connexió amb la base de dades.

3. **Per què és important l'ordre dels middlewares?**
   L'ordre importa perquè Express processa les peticions de dalt a baix: si el middleware d'errors o el parser de JSON estiguessin en el lloc equivocat, el servidor no podria llegir les dades correctament ni capturar errors, i el logging no registraria totes les peticions.

API REST i Express

1. **Com s'implementa el routing en `chatRoutes.js`?**
   El routing a `chatRoutes.js` funciona així: es crea un router amb `express.Router()` i s'assignen funcions del controlador a cada ruta segons el mètode HTTP. Per exemple, `POST /prompt` crida `registerPrompt`, `GET /conversation/:id` crida `getConversation` i `GET /models` crida `listOllamaModels`. Això permet separar les rutes de la lògica i documentar-les amb Swagger

2. **Quina és la diferència entre GET i POST i quan usar-los?**

- **GET**: sol·licitar dades sense canviar l'estat del servidor; idempotent i segur (ex.: llistar converses, obtenir estat).
- **POST**: enviar dades per crear o provocar una acció amb efectes (ex.: enviar un prompt perquè el model generi una resposta, crear una conversa).
  Escollir correctament els mètodes facilita el compliment de convencions REST, la cachabilitat i la seguretat (per exemple, evitar enviar dades sensibles per query string).

3. **Per què separar la lògica del controlador de les rutes (principis aplicats)?**
   Aquesta separació aplica el principi Single Responsibility i facilita testabilitat i mantenibilitat: les rutes només fan d'enrutadores (qualsevol canvi de camí no afecta la lògica) i els controladors encapsulen la lògica de negoci (facilitant reutilització i proves unitàries). A més simplifica la traçabilitat d'errors i la implementació de middlewares transversals.

Com gestiona el projecte els errors HTTP

1. **Com es detecten i propaguen els errors des dels controladors?**
   Els controladors capturen excepcions o validacions i poden llençar errors amb informació (missatge i codi). Si no els gestionen localment, passen l'error al `next(err)` d'Express perquè el middleware d'errors centralized el tracti i la resposta.

2. **Quin paper fa `errorHandler.js`?**
   `errorHandler.js` s'encarrega de capturar tots els errors de l'aplicació en un sol lloc, registrar-los amb el logger i enviar una resposta adequada al client segons el tipus d'error (400 per errors de dades, 500 per errors generals). Així no cal gestionar errors a cada ruta i tot queda centralitzat.

3. **Com centralitza la gestió d'errors (beneficis pràctics)?**
   Centralitzar permet: (a) tenir un punt on aplicar polítiques de seguretat (no filtrar dades sensibles), (b) afegir tracing/logging uniforme, (c) mapear errors interns a codis HTTP coherents i (d) adaptar la informació retornada segons l'entorn (més detall en dev, menys en prod). Això simplifica el debugging i la resposta a incidents.

Documentació amb Swagger

1. **Com s'integra la documentació (swagger.js i comentaris a chatRoutes.js)?**
   La documentació es crea escrivint comentaris `@swagger` a cada ruta i després `swagger.js` llegeix aquests comentaris per generar automàticament la interfície web a `/api-docs`. Així pots veure i provar l'API directament des del navegador.

2. **Quins beneficis aporta integrar documentació amb el codi?**
   - Documentació que es manté sincronitzada amb el codi evitant divergències.
   - Permet generar clients automàtics i tests.
   - Facilitza la comprensió de l'API per a altres desenvolupadors i facilita proves manuals des del navegador.
3. **Com es documenten endpoints i per què documentar paràmetres d'entrada/sortida?**
   S'usen descripcions, esquemes de requestBody i responses, tipus i exemples. Documentar paràmetres evita malentesos sobre què enviar i què rebràs (tipus, obligatorietat, format), redueix errors de contracte i facilita la integració i el debugging de clients.

Base de Dades i Models

1. **Com s'implementen relacions amb Sequelize (`Conversation` i `Prompt`)?**
   S'usa `Conversation.hasMany(Prompt)` i `Prompt.belongsTo(Conversation)`. Això crea la clau forana `ConversationId` a `Prompts` i habilita mètodes Sequelize (p. ex. `conversation.getPrompts()`, `prompt.setConversation()`) per gestionar relacions de forma objectual.
2. **Per què s'utilitza UUID com a clau primària?**
   UUID ofereix unicitat global sense dependre d'un generador seqüencial del SGBD, és útil per a entorns distribuïts, sincronització entre servidors i per evitar predicció dels IDs en APIs públiques. També evita col·lisions quan es fusionen dades de múltiples fonts.
3. **Com gestiona el projecte migracions i sincronització; riscos d'usar `sync()` en producció?**
   `sequelize.sync()` crea o adapta taules segons models; pot usar-se amb `force: true` (reescriu taules) o `alter`. Risc: en producció pot perdre dades o provocar canvis no desitjats si no es controlen. Per això és preferible usar migracions explícites (ficheros que apliquen canvis controlats) i pipelines que validin les migracions abans d'executar-les.

Logging i Monitorització

1. **Com s'implementa el logging estructurat (`logger.js`)? Quins nivells existeixen?**
   El logging s'implementa amb Winston, que registra missatges amb data, nivell i informació addicional. Hi ha un middleware per registrar cada petició HTTP

automàticament.

Els nivells principals són: **error**, **warn**, **info** i **debug**.

2. **Per què tenir diferents transports (consola, fitxer)? Com es configuren?**
   Transports separats permeten enrutar sortides diferents: consola per desenvolupament interactiu, fitxers per arxiu i posterior anàlisi, i possiblement serveis externs (Logstash, Sentry) per monitorització. Es configuren en `logger.js` definint transports i formats (JSON, text, rotació de fitxers, nivells per transport).

3. **Com ajuda el logging a debugar en producció i quina informació crítica cal registrar?**
   El logging permet reconstruir seqüències d'esdeveniments, correlacionar errors amb IDs de petició i temps, i detectar patrons de fallada. Cal registrar: timestamps, nivell, identificador de petició/usuari (correlació), endpoint i paràmetres essencials (sense dades sensibles), errors amb stack traces, temps de resposta i model/versió utilitzada (per a problemes relacionats amb l'IA).

Exercici 2 (2.5 punts)

Dins de **practica-codi** trobaràs **src/exercici2.js**

Modifica el codi per tal que, pels dos primers jocs i les 2 primeres reviews de cada joc, crei una estadística que indiqui el nombre de reviews positives, negatives o neutres.

Modifica el prompt si cal.

Guarda la sortida en el directori data amb el nom **exercici2_resposta.json**

Exemple de sortida

```
{
  "timestamp": "2025-01-09T12:30:45.678Z",
  "games": [
    {
      "appid": "730",
      "name": "Counter-Strike 2",
      "statistics": {
        "positive": 1,
        "negative": 0,
        "neutral": 1,
        "error": 0
      }
    },
    {
      "appid": "570",
      "name": "Dota 2",
      "statistics": {
        "positive": 1,
        "negative": 1,
        "neutral": 0,
        "error": 0
      }
    }
  ]
}
```

Exercici 3 (2.5 punts)

Dins de **practica-codi** trobaràs **src/exercici3.js**

Modifica el codi per tal que retorni un anàlisi detallat sobre l'animal.
Modifica el prompt si cal.

La informació que volem obtenir és:

- Nom de l'animal.
- Classificació taxonòmica (mamífer, au, rèptil, etc.)
- Hàbitat natural
- Dieta
- Característiques físiques (mida, color, trets distintius)
- Estat de conservació

Guarda la sortida en el directori **data** amb el nom **exercici3_resposta.json**

```
{
    "analisis": [
        {
            "imatge": {
                "nom_fitxer": "nom_del_fitxer.jpg",
            },
            "analisi": {
                "nom_comu": "nom comú de l'animal",
                "nom_cientific": "nom científic si és conegut",
                "taxonomia": {
                    "classe": "mamífer/au/rèptil/amfibi/peix",
                    "ordre": "ordre taxonòmic",
                    "familia": "família taxonòmica"
                },
                "habitat": {
                    "tipus": ["tipus d'hàbitats"],
                    "regioGeografica": ["regions on viu"],
                    "clima": ["tipus de climes"]
                },
                "dieta": {
                    "tipus": "carnívor/herbívor/omnívor",
                    "aliments_principals": ["llista d'aliments"]
                },
                "caracteristiques_fisiques": {
                    "mida": {
                        "altura_mitjana_cm": "altura mitjana",
                        "pes_mitja_kg": "pes mitjà"
                    },
                    "colors_predominants": ["colors"],
                    "trets_distintius": ["característiques"]
                },
                "estat_conservacio": {
                    "classificacio_IUCN": "estat",
                    "amenaces_principals": ["amenaces"]
                }
            }
        }
    ]
}
```

Exercici 4 (2.5 punts)

Implementa un nou endpoint a xat-api per realitzar anàlisi de sentiment

Haurà de complir els següents requisits

- Estar disponible a l'endpoint POST /api/chat/sentiment-analysis
- Disposar de documentació swagger
- Emmagatzemar informació a la base de dades
- Usar el logger a fitxer

Abans d'implementar la tasca, explica en el requadre com la plantejaràs i fes una proposta de json d'entrada, de sortida i de com emmagatzemaràs la informació a la base de dades.

Antes de implementar la tarea, se creará un nuevo endpoint POST dentro de la API de chat que permita realizar análisis de sentimiento sobre un texto proporcionado por el usuario. Este endpoint recibirá un texto y lo enviará al servidor Ollama utilizando un modelo de lenguaje, solicitando que clasifique el sentimiento como positivo, negativo o neutral.

Una vez obtenida la respuesta del modelo, el sistema devolverá el resultado al cliente, almacenará la información en la base de datos y registrará la operación en el fichero de logs. De esta forma, el sistema permitirá analizar opiniones, guardar los resultados y mantener un histórico de análisis.

El JSON de entrada tendrá la siguiente estructura:

text: texto que se quiere analizar.
userId: identificador del usuario (opcional).
gameId: identificador del juego o recurso relacionado (opcional).

Ejemplo:

```
{
 "text": "Me encanta este juego, es muy divertido",
 "userId": "alumno123",
 "gameId": "730"
 }
```

El JSON de salida tendrá la siguiente estructura:

```
{
 "text": "Me encanta este juego, es muy divertido",
 "sentiment": "positive",
 "timestamp": "2026-01-27T18:40:22.123Z"
 }
```

Para el almacenamiento en base de datos se utilizará una tabla llamada sentiment_analysis con los siguientes campos:

id: identificador único (UUID)
text: texto analizado
sentiment: resultado del análisis (positive, negative, neutral o error)
meta: información adicional en formato JSON (userId, gameId, etc.)
createdAt: fecha de creación
updatedAt: fecha de modificación